

ToolRL 中文精读导读：教大模型“用对工具”，关键不在示范，而在奖励怎么设计

原文：Cheng Qian, Emre Can Acikgoz, Qi He, Hongru Wang, Xiusi Chen, Dilek Hakkani-Tür, Gokhan Tur, Heng Ji (University of Illinois Urbana-Champaign, UIUC)

本导读为面向 AI 应用开发初学者的中文学习笔记：含要点讲解、关键句翻译、公式精讲与图表解读。出于版权合规，这是转化性学习材料，以转述和讲解为主，不逐字翻译原文。

论文卡片

英文标题	ToolRL: Reward is All Tool Learning Needs (工具学习只需要奖励——指好的“奖励设计”是工具学习的关键)
作者	Cheng Qian, Emre Can Acikgoz, Qi He, Hongru Wang, Xiusi Chen, Dilek Hakkani-Tür, Gokhan Tur, Heng Ji
机构	University of Illinois Urbana-Champaign (UIUC, 伊利诺伊大学厄巴纳-香槟分校)
会议	NeurIPS 2025 (神经信息处理系统大会)
引用数	262 (Semantic Scholar, 截至 2026-06-07)
代码	https://github.com/qiancheng0/ToolRL
方向	工具调用 (Tool Use) × 大模型 × 强化学习 (RL) 后训练
一句话定位	第一篇系统研究“工具调用任务该怎么设计 RL 奖励”的工作：把“用对工具”的奖励拆成工具名 + 参数名 + 参数值三块精细打分，让学习信号更密、更好学。从零 (cold start) 用 GRPO 训练，比基座模型约 +17%、比 SFT 约 +15%。

1 小白知识卡：读这篇前要懂的几个概念

这篇论文站在“大模型 + 强化学习 + 工具调用”的交叉点上，下面这几个词先弄明白，后面就顺了。

- **工具调用 / 函数调用 (tool use / function calling)**：让大模型不止“嘴上说”，还能真的去调用外部工具——比如搜索引擎、计算器、天气 API、订票接口。模型要输出一段结构化文本（通常是 JSON），写清楚“调用哪个工具（函数名）”和“传什么参数（参数名 + 参数值）”，程序再据此真正执行。

- **工具集成推理 (Tool-Integrated Reasoning, TIR)**: 本文反复出现的核心词。指模型在“思考 → 调工具 → 看工具返回结果 → 接着思考 → 再调工具”这样的多步循环里完成任务。它弥补了大模型的三大短板：知识过时、算不准、推理浅。
- **监督微调 (Supervised Fine-Tuning, SFT)**: 最常见的“喂示范”训练法——准备一堆“标准答案轨迹”，让模型照着模仿。优点是简单；缺点是容易死记硬背、遇到没见过的新工具就“水土不服”，泛化差。
- **强化学习 (Reinforcement Learning, RL)**: 不直接给标准答案，而是让模型自己生成回答（叫 rollout，采样/试探），然后用一个奖励函数给这次回答打分，好就鼓励、差就惩罚，反复迭代。本文主角就是它。
- **奖励设计 / 奖励塑形 (reward design / reward shaping)**: RL 的灵魂。奖励就是那把“打分尺”。本文最大的贡献就是论证：在工具调用任务里，这把尺子怎么刻度，比用多少数据、用哪种 RL 算法都更关键。
- **PPO 与 GRPO**: 两种主流 RL 算法。PPO (Proximal Policy Optimization, 近端策略优化) 需要额外训练一个“评论家 (critic)”网络来估计每步好坏，比较重。GRPO (Group Relative Policy Optimization, 组相对策略优化, 由 DeepSeek 提出) 不要 critic, 改成“对同一个问题采样一组回答, 组内互相比谁好谁坏”，更轻、更适合大模型。本文主要用 GRPO。
- **R1 风格 / RLVR**: DeepSeek-R1 带火的一套做法——RLVR 即“基于可验证奖励的强化学习 (RL with Verifiable Reward)”，用规则就能判定对错（比如数学答案、代码能否跑通），不需要训练人类偏好模型。本文的奖励也是这种“规则可验证”型。
- **冷启动 (cold start)**: 指直接从原始指令模型（没经过 SFT）开始做 RL。本文的一个重要发现：冷启动 GRPO 反而比“先 SFT 再 RL”泛化更好。
- **泛化 (generalization)**: 在训练时没见过的工具、编程语言或任务目标上还能做对。这是衡量“真学会了”还是“背下来了”的关键。

2 摘要全译

当前的大语言模型 (LLMs) 通常通过监督微调 (SFT) 来获得工具使用能力。然而, SFT 难以泛化到不熟悉或复杂的工具使用场景。强化学习 (RL) 的最新进展, 尤其是 R1 类模型, 已展现出有希望的推理与泛化能力。但是, 工具使用的奖励设计带来了独特的挑战: 一次可能调用多个工具、且各自带有不同的参数, 而粗粒度的奖励信号 (例如仅靠答案是否匹配) 无法提供有效学习所需的细粒度反馈。在本工作中, 我们首次系统性地研究了 RL 范式下、面向工具选择与工具应用任务的奖励设计。我们系统地探索了一系列广泛的奖励策略, 分析其类型 (types)、尺度 (scales)、粒度 (granularity) 和时间动态 (temporal dynamics)。基于这些洞见, 我们提出了一种为工具使用任务量身定制的、有原则的奖励设计, 并用它在组相对策略优化 (GRPO) 框架下训练 LLMs。在多种基准上的实证评估表明, 我们的方法带来了稳健、可扩展且稳定的训练, 相比基座模型提升了 17%、

相比 SFT 模型提升了 15%。这些结果凸显了精心设计奖励在增强 LLMs 工具使用能力与泛化性能方面的关键作用。所有代码均已发布以促进未来研究。

【小白补丁】一句话抓重点

摘要的核心就一句：别再靠“答案对不对”这种一刀切的打分了。工具调用一次要选工具、填参数，信息很多；把奖励拆细（工具名/参数名/参数值分别给分），模型才学得动。

3 逐节精读

3.1 引言：为什么 SFT 不够，要上 RL

论文从一个直觉切入：教模型用工具，主流做法是 SFT——离线造好一批“带工具调用的推理轨迹”，让模型模仿。但 SFT 有三个老毛病：泛化差、不会探索、不够灵活。

文中用图 1 举了个生动的反例：一个在“深度思考轨迹”上做过 SFT 的模型，会过度解读工具——明明该拒绝某个不合适的工具，它却硬要用；它只是机械模仿了像“but wait（等一下）”这种表面措辞，并没有真正在深度思考。这就引出了本文的根本研究问题（关键句翻译）：

“基于 RL 的训练方法，能否更好地赋予 LLMs 具备能动性（agentic）的工具使用能力？如果能，面向 TIR 的最优 RL 设计又是什么？”

作者指出，已有的 Search-R1、TORL 等工作虽然开了头，但都太窄：要么只管问答里的搜索工具，要么只管数学题里的代码工具。本文的目标是通用的工具选择与应用——覆盖多样、复杂的工具集和不同任务类型。

为什么奖励是关键？因为和“只有一个正确答案”的数学题不同，TIR 任务有多层复杂度：常常多步交互，每一步可能要调多个工具，每个工具还要仔细填参数。这种情况下，“答案匹配”这种粗粒度奖励根本不够用。作者于是从四个维度系统地研究奖励：(1) 奖励类型（reward 什么）、(2) 奖励尺度（奖多少）、(3) 奖励粒度（信号多细）、(4) 奖励动态（随训练怎么变）。

引言末尾给出三条核心洞见（直接对应后面的分析章节）：

- 更长的推理链未必更好，强行加“长度奖励”反而可能拖累性能。
- 动态的奖励尺度，能帮模型从简单行为平滑过渡到复杂行为。
- 细粒度的奖励分解，能带来更稳定、更有效的学习。

3.2 相关工作

这节梳理两条线。一条是 **LLM 的工具集成推理 (TIR)**：从早期给模型接上程序执行器、搜索引擎，到建评测基准（考察 API 选择、参数生成、泛化），再到构造高质量工具数据集、让模型自己造工具，最近开始用 RL 来进一步提升。另一条是 **LLM 上的 RL 探索**：从 RLHF（基于人类反馈的强化学习）、PPO，发展到 DPO、SimPO、GRPO，再到 DAPO、VAPO 等更新的变体。作者特别强调 GRPO 是专为 LLM 设计的——用“组内比较”替代了传统的 critic（评论家），其灵活的奖励函数能适配多种目标。本文正是在 GRPO 上扩展，做通用工具使用。

3.3 方法（一）：把 TIR 任务形式化

这部分把“多步用工具解决问题”用数学语言写清楚。给定工具集 $T = \{t_1, t_2, \dots, t_n\}$ 和用户问题 Q ，到第 k 步为止的推理轨迹记为：

$$s_k = (r_1, T_1, o_1), (r_2, T_2, o_2), \dots, (r_k, T_k, o_k)$$

逐符号拆解：

- r_i ：第 i 步模型用自然语言写的思考（reasoning）。
- $T_i \subseteq T$ ：第 i 步实际调用的工具集合（注意是“集合”——一步可以调好几个工具）。
- o_i ：执行完 T_i 后收到的观测（observation），可能含环境反馈或用户反馈。

在第 $k+1$ 步，模型要产出下一步思考 r_{k+1} 、选出工具集 T_{k+1} 并填好参数。模型策略写成 π ： $s_k \rightarrow (r_{k+1}, T_{k+1})$ ，目标是选一个工具集使即时奖励最大：

$$T_{k+1}^* = \arg \max_{T_{k+1} \subseteq T} R(s_k, T_{k+1}, o_{k+1})$$

其中 $R(\cdot)$ 就是评估“这一步调工具调得好不好”的奖励函数。把每步的即时奖励累加，就得到整条轨迹的优化目标：

$$\max_{\pi} \mathbb{E}_{\pi} \left[\sum_{k=1}^K R(s_k, T_{k+1}, o_{k+1}) \right]$$

【小白补丁】这里的关键直觉

和问答任务只看“最终答案对不对”不同，工具任务因为训练数据里每一步都有标准工具调用，所以可以给出密集的中间反馈（dense intermediate feedback）——每走一步都能打分。信号越密，模型越好学。这正是工具任务相比数学/问答的“先天优势”，本文把它用足了。

3.4 方法 (二): TIR Rollout (怎么让模型自己跑起来)

Rollout 就是“让模型自己生成思考和工具调用”的过程。作者用一段系统提示词 (图 4) 规定输出格式: 模型要用三个特殊标签来组织输出——`<think>` (思考)、`<tool_call>` (工具调用)、`<response>` (回复)。其中 `<tool_call>` 里是一条或多条 JSON, 每条含 `name` (工具名) 和 `parameters` (参数字典)。

流程是这样 (见后面图 3): 当模型输出里出现 `<tool_call>`, 系统就自动把它解析成一个个具体调用、按模型填的参数去执行, 执行结果塞进 `<obs>` 字段, 追加到对话历史里, 作为模型的交互轨迹; 如果输出里有 `<response>`, 就解析出回复同样追加进历史。值得注意的是, `<tool_call>` 和 `<response>` 不是互斥的, 可以在一次输出里同时出现 (既调工具又回话)。

3.5 方法 (三): 奖励设计——本文的核心

这是全篇的灵魂。最终奖励 R_{final} 拆成两大块: 格式奖励 R_{format} + 正确性奖励 R_{correct} 。

格式奖励 (Format Reward)。它只看“输出结构对不对”, 是个 0/1 的开关:

$$R_{\text{format}} = \begin{cases} 1, & \text{所有必需字段都出现、且顺序正确} \\ 0, & \text{否则} \end{cases}$$

也就是说, 必须老老实实把 `<think>/<tool_call>/<response>` 按规定的顺序写全, 才得这 1 分。这管的是“语法层面的合规”。

正确性奖励 (Correctness Reward)。这是精华——它把“工具调得对不对”拆成三个小项分别打分, 而不是“全对才给分”。设预测的工具调用集为 $P = \{P_1, \dots, P_m\}$, 标准答案集为 $G = \{G_1, \dots, G_n\}$:

(1) **工具名匹配 (Tool Name Matching)** ——用交并比 (IoU) 衡量选对了哪些工具:

$$r_{\text{name}} = \frac{|N_G \cap N_P|}{|N_G \cup N_P|} \in [0, 1]$$

其中 N_G 、 N_P 分别是标准答案与预测里工具名的集合。选对的工具越多、选错的越少, 分越高。

(2) **参数名匹配 (Parameter Name Matching)** ——对每个匹配上的工具, 看参数名 (字典的键 keys) 对上了多少:

$$r_{\text{param}} = \sum_{G_j \in G} \frac{|\text{keys}(P_G) \cap \text{keys}(P_P)|}{|\text{keys}(P_G) \cup \text{keys}(P_P)|} \in [0, |G|]$$

就是把“该填哪些参数”这件事也单独算分 (同样用交并比)。

(3) **参数值匹配 (Parameter Content Matching)** ——再看每个参数的值填对没有:

$$r_{\text{value}} = \sum_{G_j \in G} \sum_{k \in \text{keys}(G_j)} \mathbf{1}[P_G[k] = P_P[k]] \in \left[0, \sum_{G_j \in G} |\text{keys}(G_j)|\right]$$

$1[\cdot]$ 是指示函数：值填对记 1 分，错记 0 分。

单个匹配的总分是三项之和：

$$r_{\text{match}} = r_{\text{name}} + r_{\text{param}} + r_{\text{value}} \in [0, S_{\text{max}}], \quad S_{\text{max}} = 1 + |G| + \sum_{G_j \in G} |\text{keys}(G_j)|$$

S_{max} 就是“全对”能拿的满分。由于预测和标准答案是两个集合，作者会去找一个最优匹配（让预测的工具和标准工具一一配对、使总分最大），记最优总分为 R_{max} ，再归一化到 $[-3, 3]$ ：

$$R_{\text{correct}} = 6 \cdot \frac{R_{\text{max}}}{S_{\text{max}}} - 3 \in [-3, 3]$$

这一步把“拿到满分比例”线性映射成 $[-3, 3]$ ：全错给 -3 （惩罚），全对给 $+3$ （奖励）。

最终奖励。 两块相加：

$$R_{\text{final}} = R_{\text{format}} + R_{\text{correct}} \in [-3, 4]$$

【小白补丁】为什么这套设计是关键创新

以前的工作常用二元（对/错）或过度简化的奖励：工具调用全对才给 1 分，差一点点就 0 分。问题是工具调用太复杂——选了 3 个工具对了 2 个、参数大部分填对只错一个值，这些“部分正确”在二元奖励下全是 0 分，模型收不到任何“你在变好”的信号，学习信号变得极其稀疏。本文把它拆成工具名/参数名/参数值三层分别打分，等于给模型一个进度条：每对一点就加一点分。信号又密又有方向，自然学得又快又稳。这就是标题“Reward is All Tool Learning Needs（奖励才是工具学习真正需要的）”的底气。

3.6 方法（四）：用 GRPO 做 RL 训练

有了奖励，就用 GRPO 来更新模型。GRPO 是 PPO 的变体，核心是组内优势归一化（advantage normalization within groups）。

对每个问题 Q ，rollout 采样出一组回答，连同各自奖励构成组 $G_Q = \{A, (s_1, r_1), \dots, (s_n, r_n)\}$ （ A 是标准标注，每个 $r_i = R_{\text{format}}(s_i, A) + R_{\text{correct}}(s_i, A)$ ）。先算组内奖励的均值 μ_Q 和标准差 σ_Q ：

$$\mu_Q = \frac{1}{n} \sum_{i=1}^n r_i, \quad \sigma_Q = \sqrt{\frac{1}{n} \sum_{i=1}^n (r_i - \mu_Q)^2}$$

再给每个样本算归一化优势（normalized advantage）：

$$A_i(s_i|Q) = \frac{r_i - \mu_Q}{\sigma_Q + \eta}$$

η 是防止除零的小常数。这个式子的直觉很简单：在同一个问题的这组回答里，你比平均水平好多少。比平均好（优势为正）就鼓励，比平均差（优势为负）就压制——不需要单独训练一个 critic 来估值，省事又稳。

策略 π_θ 用标准的 PPO 裁剪目标来优化（把优势换成上面的组内归一化优势）：

$$J_{\text{GRPO}}(\theta) = \mathbb{E}_{Q \sim D} \mathbb{E}_{s_i \sim \pi_\theta} \min \left(\frac{\pi_\theta(s_i|Q)}{\pi_{\text{old}}(s_i|Q)} A_i, \text{clip} \left(\frac{\pi_\theta(s_i|Q)}{\pi_{\text{old}}(s_i|Q)}, 1 - \epsilon, 1 + \epsilon \right) A_i \right)$$

一个值得注意的工程选择：作者去掉了原版 GRPO 里对参考模型的 KL 惩罚项。这样做是为了让模型更自由地适配自定义的输出格式和奖励信号；实践中他们发现这反而收敛更快、性能相当，还简化了训练流程。

3.7 方法图解：一张图看懂整个流程

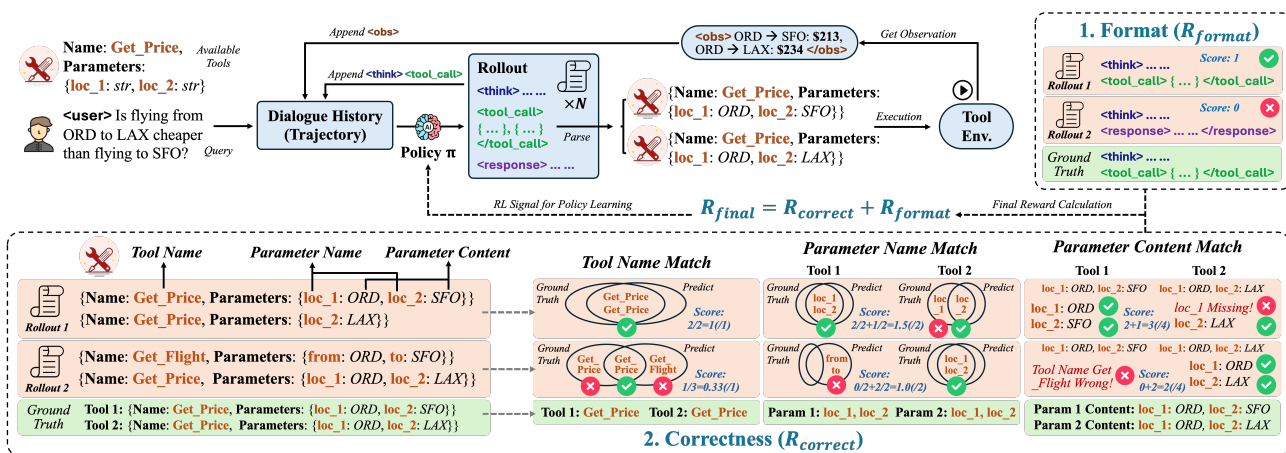


图 1: TIR rollout 与“格式奖励 + 正确性奖励”计算示意（原文图 3）。左侧绿框是 Format Reward（看 `<think>/<tool_call>/<response>` 结构是否齐全、顺序是否正确，0/1 打分）；下方蓝框是 Correctness Reward，依次做 Tool Name Match（工具名匹配）、Parameter Name Match（参数名匹配）、Parameter Content Match（参数值匹配），三块分别打分后相加再归一化到 $[-3, 3]$ 。

怎么读这张图：整张图就是把前面方法部分“可视化”了。模型拿到系统提示词和工具列表后，生成一段含 `<think>/<tool_call>/<response>` 的输出（中间那条流水线）。然后兵分两路打分：(1) 格式这一路（图右上绿色）只检查标签结构对不对，给 0 或 1；(2) 正确性这一路（图下方）把预测的工具调用和标准答案逐项对比——先比工具名（圈出选对/选错/漏选的工具），再比每个工具的参数名，最后比参数值，每一层都给出部分分数。把这些分数汇总归一化，就得到这次 rollout 的最终奖励，喂给 GRPO 去更新模型。一句话：图在说“怎么把一次工具调用，从结构到内容，细细地打一个分”。

3.8 实验设置概览

训练数据：混合三个数据集，覆盖不同复杂度——

- **ToolACE** (采样 2K 条): 通用工具使用, 教模型何时该调工具、何时直接回话。
- **Hammer (Masked)** (1K 条): 把工具名和参数名随机打乱/遮蔽, 逼模型靠“描述”而不是“背名字”来判断, 从而提升泛化、减少对特定工具的过拟合。
- **xLAM** (1K 条): 组合型, 一回合可能要调一个或多个工具, 训练模型推理工具间依赖、主动规划多样的调用动作。

多步轨迹会被拆成单步实例, 把之前的对话历史注入用户提示里保留上下文。

训练配置: 用 veRL 框架跑 GRPO; 每步采样 batch=512, 每个问题生成 4 条回答, 共训 15 个 epoch; 为鼓励探索, 去掉 KL 正则、生成温度设为 1.0。基座用 Qwen2.5-Instruct 系列 (1.5B/3B/7B) 和 Llama-3.2-3B-Instruct。

评测基准:

- **BFCL** (Berkeley Function Call Leaderboard, 伯克利函数调用排行榜): 最全面, 覆盖单步推理、多步用工具、实时执行、拒绝无关工具、同时多工具选择与应用等。
- **API-Bank**: 三档难度, 含 73 个多样复杂的 API 工具, 考多轮对话里的工具选择与应用。
- **Bamboogle**: 一个代表性问答基准, 只看最终答案准确率, 不看工具调用过程是否规范——属于“目标导向”而非“过程导向”, 用来测自由形式 (free-form) 的工具使用。

对比基线: (1) Raw 原始指令模型 (不做任何额外训练); (2) SFT (在相同的 4K 或精选 400 条数据上做 SFT, 记作 SFT4k / SFT400); (3) SFT400+GRPO / SFT400+PPO (先 SFT 再做 RL); (4) PPO (含冷启动 PPO 与 SFT 初始化的 PPO)。本文方法记作 **Ours, GRPO Cold Start** (冷启动 GRPO)。

3.9 主要结果

下表摘录三个基准上的总体准确率 (Overall Acc), 对比关键基线 (数据取自原文表 1/2/3)。可以直观看到: 在四个模型上, 冷启动 GRPO (本文) 几乎都拿到最高分。

表 1: 三大基准总体准确率对比（节选，单位%）。粗体为该模型该基准上的最佳。

基线设置	Qwen2.5-1.5B	Qwen2.5-3B	Qwen2.5-7B	Llama-3.2-3B
<i>BFCL V3 (表 1)</i>				
Raw 原始模型	19.41	33.04	41.97	22.09
SFT4k	40.67	41.97	36.53	44.16
PPO Cold Start	38.32	51.15	46.68	42.98
Ours, GRPO Cold Start	46.20	52.98	58.38	44.10
<i>API-Bank (表 2)</i>				
Raw 原始模型	30.65	51.59	62.48	40.54
SFT4k	47.07	50.92	47.07	43.89
SFT400+GRPO	61.31	62.48	54.10	56.78
Ours, GRPO Cold Start	63.15	67.00	64.66	59.13
<i>Bamboogle 问答 (表 3, 自由形式工具使用)</i>				
Raw 原始模型	20.8	52.0	69.6	34.4
SFT4k	23.2	49.6	30.4	48.8
Ours, GRPO Cold Start	44.0	60.0	72.0	52.0

这些数字说明了什么：

- **冷启动 GRPO 全面领先。**以 Qwen2.5-7B 为例，BFCL 从原始的 41.97% 提到 58.38%（约 +16 个点），且在同等数据量下比 SFT 高约 10 个绝对点。综合所有基准，论文给出的结论是：比基座模型约 +17%、比 SFT 约 +15%。
- **SFT 会“背书”反伤泛化。**一个反直觉的发现：SFT400+GRPO（先 SFT 再 RL）虽然训练奖励更高（因为 SFT 数据和 RL 数据分布对齐了），但在两个基准上泛化更差——作者推断 SFT 初始化导致记忆和过拟合，削弱了 GRPO 的泛化能力。一句话：训练奖励高 $\neq$ 泛化好。
- **GRPO 比 PPO 更稳。**PPO 冷启动有时能超过 SFT，但跨模型不稳定；GRPO 即便冷启动也能稳定拿到更高奖励、格式奖励涨得也更快。有意思的是 PPO 偏好“先 SFT 再 RL”，而 GRPO 偏好“从零冷启动”。
- **Llama-3.2-3B 提升相对小**，作者认为可能是该模型对 GRPO 式泛化的适配性较低，但仍有竞争力、在 API-Bank 上超过多数基线。

3.10 泛化研究：没见过的工具/语言/目标，照样能做对

作者在 BFCL 子集上测了两种训练时没见过的设定：

- **不熟悉的场景 (unfamiliar scenario)：**在没训过的编程语言 (Python / Java / JavaScript) 里用工具。

- **新颖的任务目标 (novel goal)**: 检测“无关工具”的能力 (该不该用工具本身)。

结果 (原文图 7): 冷启动 GRPO 在这些没见过的设定上始终最好。论文还给了两个定性例子 (表 4): (1) 用户要订电影票但没说片名, 模型主动追问澄清, 而不是瞎调; (2) 问“与 $y = 3x + 2$ 垂直的直线斜率”, 模型判断给的工具 `find_critical_points` 跟问题无关, 于是直接口算回答 $-1/3$ 、不调工具。作者把这两种行为称为涌现出的主动性 (*proactiveness*) 与元认知 (*metacognition*) ——能减少幻觉、提升效率, 是“能动智能体 (agent)”的基础特质。

3.11 分析章节：四个维度的消融实验与三条 Takeaway

这是论文的“调参智慧”部分, 逐一验证奖励的四个维度。

5.1 长度奖励 (Length Reward) 的影响。 受 R1 类模型“想得越长越好”的启发, 作者加了个鼓励 `<think>` 写长的奖励:

$$R_{\text{length}} = \min\left(\frac{L_{\text{think}}}{L_{\text{target}}}, 1\right), \quad L_{\text{target}} = 512$$

还试了随训练步数动态增大目标长度的版本 $R_{\text{dynamic}} = \min\left(\frac{L_{\text{think}}}{L_{\text{target}} \cdot (1+p)}, 1\right)$ (p 是归一化训练进度)。结果 (表 5): 长度奖励确实让回答变长了, 但并不能稳定提升性能, 在小模型上甚至明显掉点。

【小白补丁】Takeaway 1

长度奖励虽然能鼓励更长的推理链, 但并不会稳定改善任务表现, 在较小模型上甚至有害——推理更长, 对工具任务并非天生更好。过长反而引入冗余、导致“想太多 (overthinking)”。

5.2 奖励尺度 (Reward Scale) 的影响。 研究“正确性奖励”和“格式奖励”的相对权重。R1 风格通常给正确性更大权重, 以防“奖励黑客 (reward hacking)”——模型钻格式的空子拿分却没真学会。作者做了对照实验: 把正确性奖励范围从 $[-3, 3]$ 压到 $[-1, 1]$ (即 $R_{\text{correct}} = 2 \cdot \frac{R_{\text{max}}}{S_{\text{max}}} - 1$) 和格式奖励同量级, 记作“Equal Max”。结果 (表 6): 等权后多数模型略掉点, 印证了正确性应当占更大权重。在此基础上作者进一步试了动态尺度: 早期让模型先学好“格式” (简单目标), 再逐渐把重心转向“正确性” (难目标), 分两种策略:

- *Two stage* (粗粒度两阶段): 前 $s = 30$ 步把正确性奖励降到原来 $\frac{1}{3}$ 、格式保持原样; 30 步后正确性恢复、格式减半到 $[0, 0.5]$ 。
- *Dynamic* (细粒度连续): 两边随训练进度 p 连续插值, 格式尺度 $[-2 + p, 2 - p]$ 、正确性尺度 $[-2 - p, 2 + p]$, 平滑切换。

结果: 粗粒度两阶段反而掉点 (突变伤训练), 而细粒度连续过渡能提升性能。

【小白补丁】Takeaway 2

训练中逐渐调整奖励尺度（而非突变），更利于模型学习和泛化——从简单目标（学格式）到复杂目标（学正确）平滑过渡，效果最好。

5.3 奖励粒度 (Reward Granularity) 的影响。 这一节直接验证全文核心论点。作者从“最细粒度”的原始设计出发，逐步把奖励项揉成一团，造出三个越来越粗的版本：

- *Finegrained*: 工具名和参数名都改成严格精确匹配 ($r_{\text{name}} = \mathbf{1}[N_G = N_P]$, 全对才 1), 但值还分项。
- *Intermediate*: 把参数名和参数值合成一项, 要求整个参数字典精确匹配 ($r_{\text{param}} + r_{\text{value}} = \sum_{G_j} \mathbf{1}[P_G = P_P]$)。
- *Coarse*: 最粗——工具名、参数名、参数值全部纠缠, 整个工具集要和标准答案一模一样才给分 ($\mathbf{1}[G = P]$, 否则 0)。

结果（表 7 + 图 11）非常清楚：粒度越粗，模型在训练中越难拿到高奖励（信号变稀疏，“功劳分配 credit assignment”变难），最终性能也越差；而原始的最细粒度设计在各模型上都表现最好。

【小白补丁】Takeaway 3

细粒度的奖励分解提供了更丰富的学习信号，对训练更有效；而粗粒度奖励会阻碍进展、拉低最终性能。这条直接坐实了标题——把奖励拆细，是工具学习成功的关键。

3.12 结论

作者总结：他们提出了一套专为 *GRPO* 工具使用任务定制的奖励设计。从零（冷启动）训练的 *GRPO* 模型，在多个留出（held-out）基准上，稳定超过 SFT、超过“SFT 初始化的 RL”、也超过用其他 RL 算法训练的模型；并且能很好泛化到问答设定，展现出稳健的多轮交互、涌现的主动性与元认知——这些都是高效、适应性强的智能体的核心特质。对奖励类型、尺度、粒度、时间动态的深入分析，进一步揭示了奖励塑形如何影响学习与行为。最后作者点题（关键句翻译）：

“我们设想，奖励才是工具学习真正需要的（reward is all tool learning needs），而 RL 为通用且富有创造性的智能体行为提供了一条强有力的路径。”

4 核心 Takeaways 与对 AI 应用开发的启示

1. 工具学习的瓶颈在“奖励设计”，不在“数据量”。本文最颠覆的结论：与其堆更多示范数据做 SFT，不如把 RL 的奖励刻度设计好。对开发者的启示——当你的 *function-calling* 模型效果不好时，先别急着加数据，想想你的“评估/反馈信号”是不是太粗了。
2. 把“对错”拆成进度条，别用一刀切。核心技巧是把工具调用的正确性拆成工具名 / 参数名 / 参数值三层分别打分（用交并比衡量集合重叠）。这套思路可直接迁移到你自己的场景：评估 agent 时，与其“整体对/错”，不如设计部分正确也能给分的细粒度指标，无论是用于 RL 还是离线评测，都更有区分度。
3. “想得长”不等于“想得好”。盲目让模型输出更长的思维链（length reward）在工具任务上经常帮倒忙，甚至导致小模型“过度思考”。做 agent 时不要迷信长 CoT，简洁、判断准更重要。
4. 课程式、平滑的训练调度更稳。让模型先学格式、再学内容，并且权重渐变而非突变，训练更稳、泛化更好。这对任何分阶段训练（先简单后难）都是个实用经验。
5. 冷启动 RL 可能比“先 SFT 再 RL”更能泛化。SFT 容易让模型“背书”，在没见过的工具/语言上失灵；而类 R1 的 RL 更能涌现出主动追问、主动拒绝无关工具等真正“能动”的行为。做工具 agent，泛化到没见过的工具才是真本事——这正是 SFT 的软肋、RL 的强项。
6. GRPO 是性价比之选。不需要额外的 critic 网络，用“组内比较 + 优势归一化”就能稳定训练；去掉 KL 惩罚还能加速收敛。对算力有限的团队友好。

【小白补丁】给实习生的一句话总结

这篇论文教会我们：让大模型学会“用对工具”，秘诀不是喂更多标准答案，而是把打分尺子刻得更细、更密——工具名、参数名、参数值分开打分，再用 GRPO 这种轻量 RL 去优化。记住那句话：*Reward is all tool learning needs.*

参考文献（节选最关键几条，原文不译）

1. Z. Shao et al. *DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models*. 2024. (GRPO 的提出，本文 RL 算法基础)
2. D. Guo et al. *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. arXiv:2501.12948, 2025. (R1 风格 RL)
3. J. Schulman et al. *Proximal Policy Optimization Algorithms*. arXiv:1707.06347, 2017. (PPO)

4. S. G. Patil et al. *Gorilla: Large Language Model Connected with Massive APIs*. NeurIPS 2024. (BFCL 基准)
5. B. Jin et al. *Search-R1: Training LLMs to Reason and Leverage Search Engines with Reinforcement Learning*. arXiv:2503.09516, 2025.
6. X. Li, H. Zou, P. Liu. *TORL: Scaling Tool-Integrated RL*. arXiv:2503.23383, 2025.